

DOM-Graph RAG: DOM-Guided Adaptive Tiling and Budgeted Online Evidence Control for Multimodal Web-Page RAG

Alexis Firome

alexis.firome@gmail.com

Abstract

Recent visual retrieval-augmented generation (RAG) systems render documents as pixel screenshots and let a vision-language model (VLM) read tiles of the image directly, avoiding the information loss of OCR and text extraction. PixelRAG [7] popularized this idea but tiles the rendered page on a fixed pixel grid, blind to content boundaries: a table or paragraph can be sliced in half mid-tile. Separately, MAGE-RAG [9] organizes long documents into a multigranular page/element evidence graph and reads it with an online controller that activates, opens, and prunes graph nodes under a fixed action budget, but targets already-parsed long PDFs rather than rendered web pages. We present **DOM-Graph RAG**, a pipeline that replaces PixelRAG’s blind grid with tiling guided by the page’s own DOM structure: each element’s bounding box drives an adaptive tiling and patching procedure that merges undersized elements and splits oversized ones, so tile boundaries follow content boundaries by construction. The resulting tiles are organized into a MAGE-RAG-style two-level graph (page and element nodes, connected by containment and spatial reading-order edges) that we further extend across pages via hyperlink edges into a small multi-page corpus graph. We fine-tune a VLM reader contrastively on synthetically generated (tile, question) pairs with lexically-mined hard negatives, using LoRA adapters on both the vision tower and the language decoder and an InfoNCE objective. Finally, we implement an online evidence controller that evolves every node’s state through a four-state machine (INACTIVE $\rightarrow$ ACTIVE $\rightarrow$ OPENED/PRUNED) via a best-first search under a node-opening budget, so the reader receives a small, prioritized evidence subgraph instead of the whole page. We describe the full, open-sourced pipeline and its implementation choices in detail, and lay out the evaluation protocol against MAGE-RAG’s published LongDocURL and MMLongBench-Doc numbers; end-to-end benchmark results are ongoing work, reported as a work-in-progress in Section 5.

1 Introduction

Retrieval-augmented generation (RAG) over documents with rich visual layout – web pages, tables, figures, infoboxes – has long relied on text extraction: parse the HTML or run OCR, chunk the resulting text, and retrieve chunks with a dense text retriever. This pipeline discards exactly the information that layout encodes: which numbers belong to which row of a table, which caption belongs to which figure, which heading scopes which paragraph. A parallel line of work asks whether a vision-language model (VLM) can skip text extraction altogether and read the document as an image. PixelRAG [7] shows that retrieving directly over rendered page screenshots, rather than extracted text, can match or beat text-based retrieval on long, layout-heavy documents.

PixelRAG’s tiling, however, is a fixed pixel grid applied uniformly to the rendered page, independent of what is actually on it. Independent coverage of the method notes the direct consequence: the grid “slices pages by fixed pixel height, meaning a table or paragraph can get cut in half mid-tile with no awareness of content boundaries”**[TODO: cite VentureBeat coverage precisely, or drop this quote if a primary source is preferred]**. A retriever built on top of such tiles has to reconstruct content boundaries that the renderer already knew and threw away.

MAGE-RAG [9] addresses a related but distinct problem: given a long, already-segmented document, how should an agent decide *which* passages to actually read under a bounded

reasoning budget? It builds a multigranular evidence graph over page- and element-level nodes with several explicit relations (containment, reading order, layout adjacency, section hierarchy, semantic neighbors) and an online controller (its Algorithm 1) that walks the graph, activating and pruning nodes, so that only a budgeted subset of the document is ever read by the downstream model. MAGE-RAG assumes the document is already parsed into this structure; it does not address how to obtain content-aligned tiles from a rendered page in the first place.

These two systems are complementary: PixelRAG shows that pixel tiles read by a VLM are a viable unit of retrieval for visually rich documents, but tiles them blindly; MAGE-RAG shows that a graph-structured, budget-aware controller over document elements outperforms retrieving a static top- k , but assumes the elements already exist. This paper combines them by using the one artifact PixelRAG discards and MAGE-RAG never had to construct in the first place: the DOM. When the input is a rendered web page rather than a scanned PDF, the browser’s own layout engine already knows the exact bounding box of every paragraph, heading, table, and image before a single pixel is cropped.

Contributions. We introduce **DOM-Graph RAG**, an open-source pipeline (§3) that:

1. replaces PixelRAG’s blind pixel grid with *DOM-guided adaptive tiling*: elements are cropped along their own

bounding boxes, undersized elements (e.g. short list items) are merged with their vertical neighbors, oversized elements (e.g. a very tall table) are split into sub-tiles, and heading elements are never emitted alone but always paired with the content they introduce (§3.1);

2. organizes these tiles into a *multigranular graph* in the style of MAGE-RAG – page and element nodes connected by containment and a spatial reading-order relation – and extends it across documents via hyperlink edges into a small corpus graph, rather than a single isolated page (§3.2);
3. fine-tunes a general-purpose VLM into a page-element reader/retriever via *contrastive LoRA training* on synthetically generated (tile, question) pairs, using lexically-mined hard negatives, with adapters on both the language decoder and the vision tower (§3.3);
4. implements an *online evidence controller* that evolves each element node through an explicit four-state machine (INACTIVE, ACTIVE, OPENED, PRUNED) via a best-first search under a node-opening budget, generalizing MAGE-RAG’s Algorithm 1 to graphs built from live DOM structure rather than pre-parsed PDF layout (§3.4).

The full pipeline – rendering, tiling, graph construction, contrastive fine-tuning, and evidence control – is implemented and exercised end-to-end on live Wikipedia pages in an accompanying notebook, with dependency-free smoke tests over a bundled local fixture. §5 lays out the intended evaluation against MAGE-RAG’s published LongDocURL and MMLongBench-Doc baselines; we report this as work in progress rather than fabricate numbers ahead of running it.

2 Related Work

Text-based document RAG. Standard RAG pipelines chunk extracted text (fixed-length windows, sentence or section boundaries) and retrieve chunks with a dense bi-encoder [5]. Applied to visually rich documents – PDFs with tables and figures, web pages with infoboxes – this requires a layout-aware parser or OCR step, both of which are lossy and error-prone on complex layouts, and it discards the two-dimensional structure of the page (which text belongs to which cell, which figure a caption refers to) before retrieval ever runs.

Screenshot-based visual RAG. PixelRAG [7] sidesteps text extraction by rendering each page to a fixed-width screenshot and retrieving directly over image tiles with a VLM embedding model (Qwen3-VL-Embedding-2B in the reference implementation), following a broader trend of using VLMs as universal, modality-agnostic retrievers. Its tiling strategy is a blind, fixed-height pixel grid: simple and renderer-agnostic, but indifferent to where an element actually begins or ends on the page, so a table or paragraph can straddle a tile boundary. Our tiling procedure (§3.1) keeps PixelRAG’s core premise – a VLM reads image tiles rather

than extracted text – but replaces the grid with the page’s own DOM geometry.

Graph-structured evidence and online controllers for long-document QA. MAGE-RAG [9] targets long-document, multimodal QA (evaluated on LongDocURL [3] and MMLongBench-Doc [6]) by building an offline multigranular evidence graph – page and element nodes linked by containment, reading order, layout adjacency, section hierarchy, and semantic-neighbor relations – and reading it online with a budgeted controller (its Algorithm 1) that activates promising nodes, opens a bounded number of them, and prunes the rest, rather than retrieving a static top- k set once. This graph-and-controller design is the direct ancestor of our evidence controller (§3.4); the key difference is where the graph comes from: MAGE-RAG assumes an already-parsed document, whereas we construct the graph directly from the DOM of a rendered web page, which yields exact element boundaries and reading order for free but only two of MAGE-RAG’s five relation types so far (§6).

Contrastive fine-tuning and hard-negative mining. Training a bi-encoder retriever with in-batch negatives plus a handful of mined *hard* negatives – passages that are lexically or semantically close to the query but do not answer it – is standard practice in dense retrieval, e.g. DPR’s BM25-mined negatives [5]. We adopt the same idea at tile granularity, using TF-IDF cosine similarity in place of BM25 for simplicity, and generate the underlying (question, positive passage) training pairs synthetically by prompting a VLM to read each tile and produce a self-contained question, in the spirit of synthetic query generation for retriever training such as InPars [1] and Promptagator [2], adapted here to image tiles rather than text passages. We fine-tune with LoRA [4] adapters applied to both the language decoder and the vision tower of the reader VLM, following the general recipe for parameter-efficient adaptation of large pretrained transformers.

Factuality benchmarks. SimpleQA [8] measures short-form factual accuracy of language models on simple, unambiguous questions; we include it in our evaluation protocol (§5) as a sanity check that evidence-controlled reading does not regress on questions that do not require multi-hop or layout reasoning.

3 Method

Figure 1 gives an overview. A page is rendered once with a headless browser; its DOM geometry drives an adaptive tiling step (§3.1); tiles are organized into a two-level graph, optionally spanning several hyperlinked pages (§3.2); a VLM reader is contrastively fine-tuned to score (question, tile) pairs (§3.3); and, at query time, an online controller walks the graph under a budget to decide which nodes the reader actually opens (§3.4).

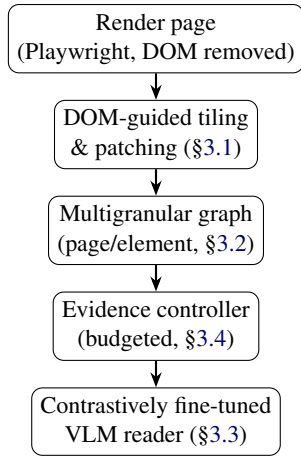


Figure 1: Pipeline overview. Rendering and tiling happen once per page; the evidence controller runs once per query, deciding which graph nodes the reader opens.

3.1 DOM-Guided Adaptive Tiling

Rendering. We load the page in headless Chromium at a fixed viewport width (2048 px) and remove interface chrome before capture – navigation bars, footers, headers, site notices, and Wikipedia-specific furniture (`.navbox`, `.mw-editsection`, ...) – via a fixed CSS selector list, then take a full-page screenshot. This mirrors PixelRAG’s rationale for stripping non-content chrome before tiling, but the removal happens in the live DOM rather than post-hoc on pixels.

Element extraction. For a configurable set of tags (headings `h1`–`h4`, `p`, `table`, `figure`, `img`, `ul/ol`, `blockquote`, `pre`), we read each matching element’s absolute bounding box (`getBoundingClientRect`, offset by scroll position) directly from the rendered DOM – the exact geometry the layout engine already computed, with no separate layout-analysis model needed. Hyperlinks are additionally collected from `<p>` elements only (never from tables or reference lists), each resolved to an absolute URL; these seed the cross-page edges of §3.2.

Adaptive tiling and patching. Elements are processed in top-to-bottom order and turned into image crops of the full-page screenshot under three rules, so that tile boundaries follow content boundaries rather than a fixed pixel pitch:

- **Heading pairing.** A heading element is never emitted as a standalone tile – alone, it carries no answerable content. It is held pending and merged with the very next element into a single tile spanning their union bounding box (e.g. a tile tagged `h2+p` for a section header immediately followed by its first paragraph).
- **Merging (undersized elements).** Non-heading elements shorter than a minimum height (80 px) are buffered and their union bounding box accumulated until the buffered span reaches that minimum, at which

point the buffer is flushed as one merged tile. This avoids emitting a flood of near-empty micro-tiles for, e.g., short list items.

- **Patching (oversized elements).** An element taller than a maximum tile height (2048 px, matching the render width) is sliced into $\lceil h/h_{\max} \rceil$ vertical sub-tiles of equal height, so a very tall table or figure still yields tiles a VLM can read at native resolution rather than one enormous, aliased crop.

Each tile retains a pointer back to its source DOM element(s) (tag, concatenated text preview, and any collected hyperlinks), used both for the graph’s text attributes (§3.2) and, later, as ground truth for judging whether a mined negative tile accidentally contains the answer (§3.3).

This directly targets PixelRAG’s failure mode: a table that would straddle two fixed-height grid cells is instead emitted as one tile (or patched into content-aligned sub-tiles if it is unusually tall), and a short caption is merged with its neighbor rather than left as an orphan sliver.

3.2 Multigranular Graph Construction

Following MAGE-RAG, tiles are organized into a two-level directed graph with one `page` node and one `element` node per tile, rather than a flat list of retrieval units. We currently instantiate three of MAGE-RAG’s five relation types:

- **contains** (`page` → `element`): every element belongs to its page.
- **reading_order** (`elementi` → `elementi+1`): a spatial reading-order heuristic, *not* raw DOM order. Elements are grouped into visual rows by vertical-overlap tolerance (20 px), each row is sorted left-to-right, and rows are concatenated top-to-bottom. This matters because DOM order and visual order diverge for CSS-positioned layouts (e.g. a sidebar infobox that appears after the lead paragraph in the DOM but is rendered beside it).
- **links_to** (`element` → `external_page`): hyperlinks collected from the first $k=3$ text tiles (in reading order) that carry any – typically the lead section – are kept as edges to placeholder external-page nodes; links beyond that budget are dropped to bound the branching factor of the corpus crawl below.

Every element node is initialized with a `state` attribute set to `INACTIVE`; this is the state the online controller of §3.4 evolves at query time. It is set once, at construction time, and is otherwise inert during graph building.

Corpus graph. A single page’s graph can be extended into a small multi-document corpus: starting from a seed page, we crawl up to m pages reachable via its `links_to` edges (one hop, non-recursive, to keep the crawl from exploding combinatorially), build each linked page’s own two-level graph independently, and merge them by namespacing every node with a slug derived from its page URL before composing the graphs – rewiring each seed-page placeholder external-page

```

<page url="..." title="...">
  <element id="tile_0000" tag="merged"
    state="inactive">...</element>
  <element id="tile_0003" tag="p"
    state="opened">Gunn was born
    on October 11, 1903...</element>
  <element id="tile_0004" tag="h2+p"
    state="inactive">...</element>
  ...
</page>

```

Figure 2: Excerpt of the XML rendering of a page graph after the evidence controller has run on the query “When and where was George Purnell Gunn born?”. Only the paragraph tile carrying the birth date/place has been opened; the rest remains inactive or pruned.

node onto the now-crawled page’s real page node. The result is a single graph spanning several pages, with `links_to` edges as the only cross-page connective tissue; this is the substrate the evidence controller (§3.4) and, eventually, a multi-page-aware retriever would operate over.

Structured rendering. For any (sub-)graph, we serialize the page node and its directly contained elements to XML – one `<element id="..." tag="..." state="...">text</element>` per node – as the structured multimodal input handed to the reader, in the same spirit as MAGE-RAG’s structured-rendering step. Figure 2 shows an excerpt after the controller of §3.4 has run.

3.3 Contrastive Reader Fine-Tuning

To turn a general-purpose VLM into a tile-level reader that can also score how well a tile answers a query, we fine-tune it contrastively on synthetically generated data, in three stages.

Synthetic QA generation. For every tile, we prompt the VLM with the tile image alone and ask it to produce one self-contained, factual question whose answer is visible *only* in that tile (a date, a name, a number, a caption – not a vague “what is this image about?”), or to answer `SKIP` if the tile carries no exploitable content (e.g. purely decorative). This follows the recipe of synthetic query generation for retriever training (e.g. InPars, Promptagator [1, 2]), applied here to image tiles instead of text passages. Every tile crop is persisted to disk regardless of whether it yielded a question, so that tiles which failed to produce a usable QA pair remain available as candidate hard negatives in the next stage.

Hard-negative mining. For each generated question, we rank every other tile from the *same page* by TF-IDF cosine similarity between the question and the tile’s text preview, then keep the top $n=2$ that (a) are not the positive tile and (b) do not already contain the answer string verbatim (to avoid manufacturing false negatives – e.g. a summary tile that

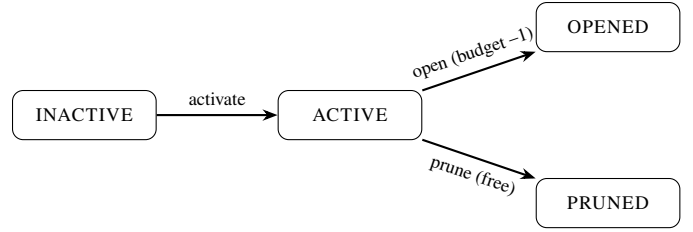


Figure 3: The controller’s state machine. Opening a node consumes one unit of the action budget and may activate unexplored `reading_order` neighbors; pruning is free and also applied to any node still ACTIVE once the budget is exhausted.

happens to repeat a figure from the body). These lexically-close-but-non-answering tiles are cheaper to obtain than a learned hard-negative miner and, unlike randomly sampled negatives, are not already trivially distinguishable from the positive, which is what makes them useful contrastive signal.

Contrastive LoRA fine-tuning. Qwen2-VL is a generative, text-to-text model, not a similarity encoder. We turn it into one by prompting it with a short, fixed instruction (“summarize the above in one word”) after either the question text or a tile image, and taking the last-token hidden state of the final layer as that input’s embedding – a technique reported in recent work on repurposing generative VLMs as universal encoders[**TODO: confirm and cite the exact reference before the final version**]. We LoRA-adapt ($r=16$, $\alpha=32$, dropout 0.05) both the language decoder’s attention projections and the vision tower’s projections, rather than the decoder alone, since the task requires jointly refining what the model attends to *in the image* and how it interprets *the question*. Given a batch of B questions, each with one positive tile and K mined hard negatives, we compute normalized question and tile embeddings and minimize a temperature-scaled ($\tau=0.05$) InfoNCE loss with cross-entropy over the $B(1+K)$ tiles in the batch – so each question’s positive competes against its own mined negatives *and*, in batch, against every other question’s positive and negatives.

3.4 Online Evidence Controller

Given a query, the controller of §3.2 decides which element nodes the reader actually reads, rather than handing it the whole page (or a static top- k tile list). It evolves each element node’s `state` through the four-state machine of Figure 3, generalizing MAGE-RAG’s Algorithm 1 to graphs built as in §3.2.

Relevance scoring. Every element node is scored against the query by TF-IDF cosine similarity between the query and the node’s text (the same lightweight, dependency-light mechanism as hard-negative mining, §3.3, with a pure-Python token-overlap fallback if `scikit-learn` is unavailable). This is a deliberate placeholder: the natural upgrade, once the

Algorithm 1 Budgeted best-first evidence control

Require: graph G , query q , budget B , seed count k , thresholds $\theta_{\text{open}}, \theta_{\text{act}}$

- 1: score every element node by $\text{sim}(q, \text{text}(n))$
- 2: activate the k highest-scoring inactive nodes $\triangleright$ seed
- 3: $b \leftarrow 0$
- 4: **while** $b < B$ and some node is ACTIVE **do**
- 5: $n \leftarrow \arg \max \text{score over ACTIVE nodes}$
- 6: **if** $\text{score}(n) < \theta_{\text{open}}$ **then**
- 7: $n \leftarrow \text{PRUNED}$ $\triangleright$ free
- 8: **else**
- 9: $n \leftarrow \text{OPENED}$; $b \leftarrow b + 1$; record its text as evidence
- 10: **for** each inactive `reading_order` neighbor n' of n **do**
- 11: **if** $\text{score}(n') \geq \theta_{\text{act}}$ **then**
- 12: $n' \leftarrow \text{ACTIVE}$
- 13: **end if**
- 14: **end for**
- 15: **end if**
- 16: **end while**
- 17: prune every node still ACTIVE $\triangleright$ finalize
- 18: **return** concatenation of OPENED nodes' text, in reading order

reader of §3.3 is fine-tuned, is to score nodes with its learned question–tile similarity instead of a lexical proxy (§6).

Policy. Algorithm 1 summarizes the loop. It (1) seeds the frontier by activating the k highest-scoring inactive nodes; (2) repeatedly takes the highest-scoring *active* node – pruning it for free if its score is below an opening threshold, otherwise opening it (consuming one unit of the budget B) and, only then, activating any of its still-inactive `reading_order` neighbors whose own score clears an activation threshold; and (3) once the budget is exhausted or no active node remains, prunes whatever is still ACTIVE, so that no node is left in a transient state. Because opening a node's neighbors are only considered *after* that node has actually been opened, the frontier grows along reading order conditioned on what has already proven relevant, rather than being fixed once from a single global ranking pass – this is what distinguishes the controller from static top- k retrieval over the same relevance scores (an ablation we report in §5).

Output. The concatenation of OPENED nodes' text, in reading order, is the final evidence passed to the downstream reader – a small, prioritized subset of the page rather than the full document, with the controller's action log (one entry per state transition) available for inspection or visualization.

4 Implementation

The pipeline is implemented in Python and released as an open, modular package: rendering and geometric extraction

Benchmark	Metric	MAGE-RAG
LongDocURL	Accuracy	52.75
MMLongBench-Doc	Accuracy / F1	53.26 / 51.19

Table 1: Published baseline numbers from MAGE-RAG [9], the reference our evidence controller (§3.4) generalizes. DOM-Graph RAG's results on the same benchmarks are ongoing work.

via headless Chromium (Playwright); tiling and patching over Pillow crops of a full-page screenshot; graph construction and corpus composition with `networkx`; the reader VLM served either through Ollama (`qwen3-v1`, the default, requiring no GPU or heavy dependencies) or through Hugging Face transformers (`Qwen2-VL-7B-Instruct`) when LoRA fine-tuning or direct access to hidden states is needed; hard-negative mining with `scikit-learn`'s TF-IDF and cosine similarity (with a dependency-free fallback); and contrastive LoRA fine-tuning with `peft` and `torch`, LoRA target modules discovered by introspecting the loaded model rather than hard-coded, so the code does not silently no-op if an internal module-naming convention changes across transformers versions. An accompanying Jupyter notebook runs every stage end-to-end against live Wikipedia pages, including interactive graph visualizations (state-colored, with tile images embedded in node tooltips) built with `pyvis`. Two dependency-light smoke tests exercise the geometric pipeline (DOM extraction through XML export) and the synthetic-QA/mining/dataset-assembly stages against a bundled local HTML fixture, requiring neither network access nor a real VLM or GPU, so the core pipeline's correctness can be checked in CI without any external service.

5 Evaluation Protocol and Status

This section describes the intended evaluation and reports its current status honestly: the pipeline of §3 is implemented and exercised end-to-end (rendering through evidence control, on live pages), but we have not yet run the full benchmark suite below. We report this as work in progress rather than fill the section with numbers ahead of the actual runs.

Benchmarks. We plan to evaluate on the same long-document, multimodal QA benchmarks used by MAGE-RAG – LongDocURL [3] and MMLongBench-Doc [6] – plus SimpleQA [8] as a short-form factuality sanity check that layers of graph pruning do not regress simple, single-fact questions. Table 1 reproduces MAGE-RAG's published numbers on the first two benchmarks as the reference to match or beat; DOM-Graph RAG's own results on these benchmarks are pending (see status paragraph below), so no comparison row is filled in yet.

Planned ablations. Three ablations isolate each contribution of §3: (a) *tiling*: DOM-guided tiling vs. a blind fixed-height pixel grid over the same rendered pages, with the reader model held fixed, to isolate the effect of content-aligned tile boundaries; (b) *evidence control*: the budgeted controller of §3.4 vs. a static top- k retrieval baseline using the same relevance scores, at matched budget/ k , to isolate the effect of best-first frontier expansion over a fixed ranking; (c) *contrastive fine-tuning*: the base VLM vs. the LoRA-fine-tuned reader of §3.3, to isolate the effect of tile-level contrastive adaptation.

Status. Phases 1–3 of §3 (rendering, tiling, graph construction and export, contrastive fine-tuning pipeline, and evidence controller) are implemented, unit- and smoke-tested, and demonstrated end-to-end on live Wikipedia pages (§4). Running the benchmarks above and the ablations requires compute and time beyond the scope of this draft; we will complete this section, with full results tables, in a subsequent revision.

6 Limitations

Relation coverage. Our graph currently instantiates three relation types (`contains`, `reading_order`, `links_to`) against MAGE-RAG’s five; we are missing *layout adjacency* and *section hierarchy* edges (both derivable from DOM geometry and heading structure without new dependencies) and *semantic-neighbor* edges (which require an embedding model – naturally, the fine-tuned reader of §3.3 once available).

Nested-element overlap. Elements that nest in the DOM (e.g. an `<img>` inside a `<figure>`, a `<p>` inside a cell of a `<table>`) are extracted independently and can therefore produce overlapping tiles; we do not yet deduplicate an element whose bounding box is already contained in a parent element’s tile.

Lexical relevance scoring in the controller. The evidence controller (§3.4) scores nodes with TF-IDF cosine similarity rather than a learned embedder, so its notion of relevance is purely lexical and will miss paraphrases or cross-modal cues that only the (image-reading) contrastively fine-tuned reader would catch. Wiring the reader’s own question–tile similarity into the scoring step is the natural next iteration, and would also let the controller reuse a single learned notion of relevance across mining (§3.3) and control (§3.4) instead of two different mechanisms.

Single-page scope for mining and (mostly) for control. Hard-negative mining operates within one page at a time, since tile identifiers are unique only at page scope; mining negatives *across* the pages of a corpus graph (§3.2) is a

natural extension that would require reusing the same slug-prefixing scheme used to merge per-page graphs. The evidence controller itself already operates over whatever graph it is given, including a multi-page corpus graph, but we have not yet evaluated it at that scale, nor extended its neighbor-propagation step to cross the `links_to` relation into newly-activated pages.

Shallow, non-recursive crawl. The corpus graph follows hyperlinks one hop deep from a seed page, bounded to a handful of linked pages, to avoid the combinatorial blow-up of a recursive web crawl; this is sufficient to exercise cross-page graph structure but is far short of the scale a genuinely multi-document retrieval setting would require.

7 Conclusion

We described DOM-Graph RAG, a pipeline that grounds two recent ideas in visual and graph-structured RAG – Pixel-RAG’s screenshot tiles and MAGE-RAG’s budgeted graph controller – in the one source of structure a rendered web page already provides for free: its DOM. DOM-guided adaptive tiling replaces a blind pixel grid with content-aligned tiles; a multigranular page/element graph, extendable across hyperlinked pages, replaces a flat tile list; a contrastively fine-tuned VLM reader replaces a generic off-the-shelf one; and a four-state, budget-bounded evidence controller replaces retrieving either the whole page or a static top- k . Every stage is implemented, tested, and demonstrated end-to-end on live pages. The immediate next step is Phase 4: running the LongDocURL, MMLongBench-Doc, and SimpleQA evaluation and the three ablations laid out in §5, and, in parallel, closing the gaps noted in §6 – most importantly, replacing the controller’s lexical relevance scoring with the fine-tuned reader’s own learned similarity once it is trained at scale.

References

- [1] Luiz Bonifacio, Hugo Abonizio, Marzieh Fadaee, and Rodrigo Nogueira. Inpars: Data augmentation for information retrieval using large language models, 2022.
- [2] Zhuyun Dai, Vincent Y. Zhao, Ji Ma, Yi Luan, Jianmo Ni, Jing Lu, Anton Bakalov, Kelvin Guu, Keith B. Hall, and Ming-Wei Chang. Promptagator: Few-shot dense retrieval from 8 examples, 2022.
- [3] Chao Deng et al. Longdocurl: a comprehensive multi-modal long document benchmark integrating understanding, reasoning, and locating, 2024. Author list and venue to verify precisely before final submission.
- [4] Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. Lora: Low-rank adaptation of large language models, 2021.

- [5] Vladimir Karpukhin, Barlas Oğuz, Sewon Min, Patrick Lewis, Ledell Wu, Sergey Edunov, Danqi Chen, and Wentaoh Yih. Dense passage retrieval for open-domain question answering. In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2020.
- [6] Yubo Ma, Yuhang Zang, et al. Mmlongbench-doc: Benchmarking long-context document understanding with visualizations, 2024. Author list to verify precisely before final submission.
- [7] Yichuan Wang et al. Pixelrag: Web screenshots beat text for retrieval-augmented generation, 2026. UC Berkeley / BAIR / Berkeley NLP. Code: github.com/StarTrail-org/PixelRAG. Author list as listed on arXiv; verify full author list before final submission.
- [8] Jason Wei et al. Measuring short-form factuality in large language models, 2024. OpenAI. Author list to verify precisely before final submission.
- [9] Yilong Zuo et al. Multigranular adaptive graph evidence for agentic multimodal rag in long-document qa, 2026. Code: github.com/laonuo2004/MAGE-RAG. Author list as listed on arXiv; verify full author list before final submission.